

Jeevanth S

23MID0019

1. Excel using Openpyxl

```
In [47]: # a. Installing required Package
!pip install openpyxl
import openpyxl
```

Defaulting to user installation because normal site-packages is not writeable

Requirement already satisfied: openpyxl in /opt/anaconda/lib/python3.13/site-packages (3.1.5)

Requirement already satisfied: et-xmlfile in /opt/anaconda/lib/python3.13/site-packages (from openpyxl) (1.1.0)

```
In [48]: # b. Load an Excel Workbook
wb = openpyxl.Workbook()
```

```
In [49]: # c. Use the active attribute to call an active worksheet
ws = wb.active
ws.title = "SalesData"

# Adding sample data for demonstration
ws['A1'] = "Product"
ws['B1'] = "Sales"
ws['C1'] = "Region"
ws['D1'] = "Target Met" # For question e

ws.append(["Laptop", 1200, "East", "Yes"])
ws.append(["Phone", 800, "West", "No"])
ws.append(["Tablet", 300, "East", "Yes"])
```

```
In [50]: # d. Count the number of rows and columns in the worksheet
total_rows = ws.max_row
total_columns = ws.max_column
print(f"Total Rows: {total_rows}, Total Columns: {total_columns}")
```

Total Rows: 4, Total Columns: 4

```
In [51]: # e. Read data from a specific cell D1
cell_d1_value = ws['D1'].value
print(f"Data in D1: {cell_d1_value}")
```

Data in D1: Target Met

```
In [52]: # f. Read data from a row from First column to nth Column (e.g., Row 2, column=col)
n = ws.max_column
row_data = [ws.cell(row=2, column=col).value for col in range(1, n + 1)]
print(f"Row 2 data (1 to {n}): {row_data}")
```

Row 2 data (1 to 4): ['Laptop', 1200, 'East', 'Yes']

```
In [53]: # g. Assign a value to a cell using cell name
ws['K1'] = "Status Report"
```

```
In [54]: # h. Assign a value to cell using row and column positions (Row 2, Column
ws.cell(row=2, column=10, value="Processed")
```

```
Out[54]: <Cell 'SalesData'.J2>
```

```
In [55]: # i. Save the excel file
wb.save("business_data.xlsx")
```

```
In [56]: #j. Write a loop to calculate the sum of all values in a row, assign the
#and name the cell

ws.append([10, 20, 30, 40, 50])
target_row = 5

# Write a loop to calculate the sum
row_sum = 0
for col in range(1, ws.max_column + 1):
    val = ws.cell(row=target_row, column=col).value
    if isinstance(val, (int, float)):
        row_sum += val

# Assign total value to a cell and name the cell
result_cell = ws.cell(row=target_row, column=ws.max_column + 2, value=row_sum)
print(f"Calculated sum {row_sum} and assigned it to cell: {result_cell.coordinate}")

wb.save("business_data.xlsx")
```

Calculated sum 150 and assigned it to cell: M5

```
In [57]: #k. Adding formulas to specific cells
ws['B6'] = "=AVERAGE(B2:B4)"
ws['B7'] = '=COUNTIF(D2:D4, "Yes")'
ws['B8'] = '=SUMIF(C2:C4, "East", B2:B4)'
ws['B9'] = '=CEILING(B6, 100)'

print("Formulas injected successfully into cells B6, B7, B8, and B9.")
wb.save("business_data.xlsx")
```

Formulas injected successfully into cells B6, B7, B8, and B9.

```
In [58]: #l. Rename a sheet
ws.title = "Updated_SalesData"

# Duplicate a sheet
duplicated_ws = wb.copy_worksheet(wb["Updated_SalesData"])
duplicated_ws.title = "Copied_Sheet"

# Create a temporary sheet to demonstrate deletion
temp_sheet = wb.create_sheet(title="ToBeDeleted")
print("Sheets before deletion:", wb.sheetnames)

# Delete a sheet
wb.remove(wb["ToBeDeleted"])
print("Sheets after deletion:", wb.sheetnames)

wb.save("business_data.xlsx")
```

Sheets before deletion: ['Updated_SalesData', 'Copied_Sheet', 'ToBeDeleted']

Sheets after deletion: ['Updated_SalesData', 'Copied_Sheet']

```

In [59]: #m. Add a bar chart

from openpyxl.chart import BarChart, Reference
# i. Load the sheet
chart_ws = wb["Updated_SalesData"]

# Set up clean data for charting
chart_ws['A12'] = "Month"
chart_ws['B12'] = "Revenue"
chart_data = [
    ["Jan", 2500],
    ["Feb", 3100],
    ["Mar", 2800],
    ["Apr", 4200]
]
for row in chart_data:
    chart_ws.append(row)

# ii. Import Reference to set boundaries for the bar chart data
data_ref = Reference(chart_ws, min_col=2, max_col=2, min_row=12, max_row=15)

# iii. Set the parameters for chart categories
cats_ref = Reference(chart_ws, min_col=1, max_col=1, min_row=13, max_row=16)

# iv. Create a Bar chart object
chart = BarChart()
chart.type = "col" # Vertical bar chart
chart.style = 10

# v. Add data
chart.add_data(data_ref, titles_from_data=True)

# vi. Set categories
chart.set_categories(cats_ref)

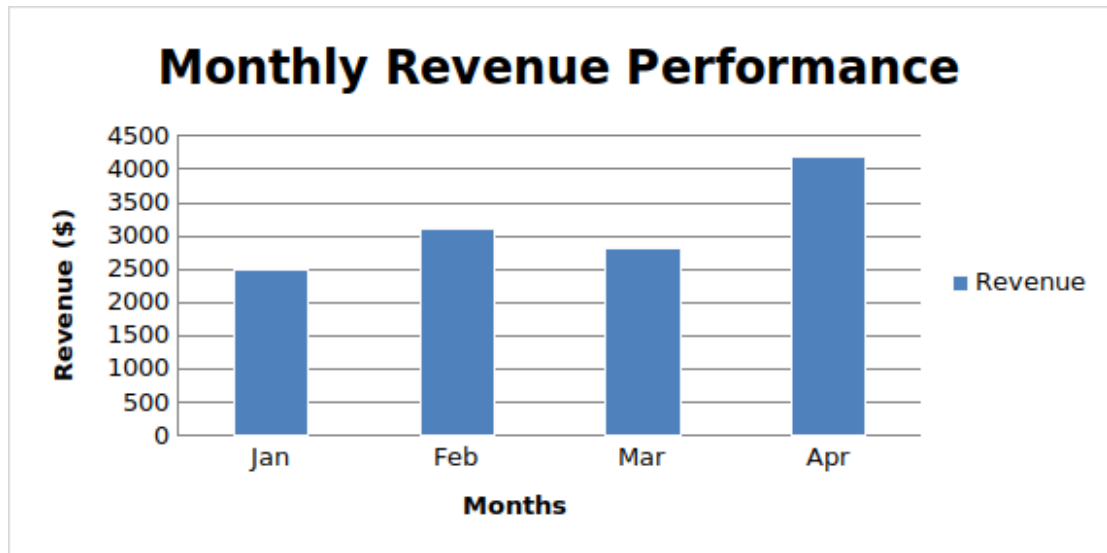
# vii. Set the title
chart.title = "Monthly Revenue Performance"
chart.x_axis.title = "Months"
chart.y_axis.title = "Revenue"

# viii. Add the chart object to a specific location on the sheet
chart_ws.add_chart(chart, "E12")

# ix. Save the Excel file
wb.save("business_data_with_chart.xlsx")
print("Workbook saved with Bar Chart added at E12.")

```

Workbook saved with Bar Chart added at E12.



2. Excel using Pandas

```
In [73]: import pandas as pd

dummy_data = {
    'Product': ['Laptop', 'Phone', 'Tablet', 'Monitor', 'Keyboard'],
    'Sales': [34000, 16000, 15000, 31000, 2500],
    'Region': ['East', 'West', 'East', 'North', 'South'],
    'Rating': [4.5, 4.2, 3.8, 4.0, 4.6]
}
pd.DataFrame(dummy_data).to_excel("sales_data.xlsx", index=False)
```

```
In [74]: # a. Load the Excel sheet to a data frame
df = pd.read_excel("sales_data.xlsx")
```

```
In [75]: # b. Inspect the data with df.head(), df.info(), df.describe()
print("--- df.head() ---")
print(df.head())

print("\n--- df.info() ---")
df.info()

print("\n--- df.describe() ---")
print(df.describe())
```

```

--- df.head() ---
   Product  Sales Region  Rating
0   Laptop  34000   East    4.5
1    Phone  16000   West    4.2
2   Tablet  15000   East    3.8
3  Monitor  31000  North    4.0
4  Keyboard   2500  South    4.6

--- df.info() ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ---
 0  Product    5 non-null      object
 1  Sales      5 non-null      int64
 2  Region     5 non-null      object
 3  Rating     5 non-null      float64
dtypes: float64(1), int64(1), object(2)
memory usage: 292.0+ bytes

--- df.describe() ---
           Sales      Rating
count    5.000000  5.000000
mean    19700.000000  4.220000
std     12882.158204  0.334664
min      2500.000000  3.800000
25%     15000.000000  4.000000
50%     16000.000000  4.200000
75%     31000.000000  4.500000
max     34000.000000  4.600000

```

```

In [76]: # c. Print the data by a specific Column name
print("\n--- Printing 'Sales' Column ---")
print(df['Sales'])

```

```

--- Printing 'Sales' Column ---
0    34000
1    16000
2    15000
3    31000
4     2500
Name: Sales, dtype: int64

```

```

In [77]: # d. Print the data from the first five rows
print("\n--- First 5 Rows ---")
print(df.iloc[:5]) # Or df.head(5)

```

```

--- First 5 Rows ---
   Product  Sales Region  Rating
0   Laptop  34000   East    4.5
1    Phone  16000   West    4.2
2   Tablet  15000   East    3.8
3  Monitor  31000  North    4.0
4  Keyboard   2500  South    4.6

```

```

In [78]: # e. Filter the data in a specific column with a specific condition (e.g.
print("\n--- Filtered Data (Sales > 400) ---")
filtered_df = df[df['Sales'] > 400]
print(filtered_df)

```

```

--- Filtered Data (Sales > 400) ---
   Product  Sales Region  Rating
0   Laptop  34000   East    4.5
1    Phone  16000   West    4.2
2   Tablet  15000   East    3.8
3  Monitor  31000  North    4.0
4  Keyboard   2500  South    4.6

```

```

In [79]: # f. Add a new column from an existing column copying values that satisfy
# Example: Create 'High_Value_Sales', copy 'Sales' value if it's > 400, o
import numpy as np
df['High_Value_Sales'] = np.where(df['Sales'] > 400, df['Sales'], np.nan)

```

```

In [80]: # g. Update a value
# Example: Change the 'Rating' of 'Tablet' (row index 2) to 4.1
df.at[2, 'Rating'] = 4.1

```

```

In [81]: # h. Print the Data Frame
print("\n--- Updated Data Frame ---")
print(df)

```

```

--- Updated Data Frame ---
   Product  Sales Region  Rating  High_Value_Sales
0   Laptop  34000   East    4.5             34000.0
1    Phone  16000   West    4.2             16000.0
2   Tablet  15000   East    4.1             15000.0
3  Monitor  31000  North    4.0             31000.0
4  Keyboard   2500  South    4.6              2500.0

```

```

In [82]: # i. Load the updated Data Frame back to Excel
df.to_excel("updated_sales_data.xlsx", index=False)
print("\nSuccess: Saved updated DataFrame to 'updated_sales_data.xlsx'")

```

Success: Saved updated DataFrame to 'updated_sales_data.xlsx'

j. Print the screenshot of updated Excel sheet

	A	B	C	D	E	F
1	Product	Sales	Region	Rating	High_Value_Sales	
2	Laptop	34000	East	4.5	34000	
3	Phone	16000	West	4.2	16000	
4	Tablet	15000	East	4.1	15000	
5	Monitor	31000	North	4	31000	
6	Keyboard	2500	South	4.6	2500	
7						

In []: